Time Varying Parameters

Ram Prasad

4/25/2025

Summary

This notebook outlines and tests procedures for estimating linear models with time varying parameters. In part one, I compare the canonical state space model to kernel regression using a simulation study. In part two, I outline additional estimation methods that extend kernel regression and perform a separate simulation. The structure for part one is as follows.

$$y_t = x_t'\beta_t + \epsilon_t, t = 1...T$$

Structure

-> 1. Motivating Example w/CAPM: Fit the CAPM using daily AAPL and SP500 returns over a 10 year period and using rolling OLS. Compare whole sample to rolling results and test for parameter stability.

-> 2. Data Simulation: Simulate market returns using a simple GBM process calibrated with historical data. Resample risk free returns non-parametrically. Simulate random walks for the time-varying alpha and beta parameters, without drift, and use the CAPM's linear combination of parameters and market returns to produce simulate excess stock returns.

-> 3. a) Procedures: Brief mathematical exposition for the Kalman Filter and Kernel regression.

-> 3. b) Testing: For each simulated set of excess returns, market returns (exogenous), and time varying parameters, estimate the parameters using KF and kernel regression, and compute error measures using simulated TVP as true values. Compare distributions of errors for time varying alpha and beta.

Standard imports.

```
import pandas as pd
import numpy as np
import random
import statsmodels.api as sm
import matplotlib.pyplot as plt
import math
```

```
import time

from scipy.stats import norm
from statsmodels.stats.diagnostic import breaks_cusumolsresid
from statsmodels.iolib.summary2 import summary_col
from scipy import stats
from sklearn.metrics import r2_score, mean_squared_error, mean_absolute_percentage_error
```

Motivation

Derive CAPM to make the argument for time varying beta. In expected return-beta form, the CAPM is:

$$E(R^i - R^f) = \beta_{i,R^W}\left[E(R^W) - R^f\right]$$
$$\beta_{i,R^W} = \frac{\mathrm{Cov}(R^i, R^W)}{\mathrm{Var}(R^W)}$$

for return R^i, R^w the return on the total wealth portfolio, often proxied using the return on an stock index, R^f the risk-free rate, and β_{i,R^w} the regression coefficient of R^i on R^W. The CAPM states that an asset's expected excess return equals β_{i,R^w}, the amount of asset i's exposure to the market factor's risk, multiplied by the factor risk premium or equivalently the market price of risk, $\lambda_{W} = E(R^W) - R^f$.

To test the model's hypothesis, that expected returns are linear in betas, perform a time series regression using any excess return i as a test asset:

$$R_t^i - R^f = \alpha_i + \beta_i(R_t^m - R^f) + \epsilon_t^i$$

and evaluate whether the pricing error and regression intercept, α_i, is statistically indistinguishable from zero. This should be the case. Second stage is cross sectional regression, which I skip because I only use one asset.

Empirical tests of the CAPM and other factor models often assume time-invariant parameters, as a consequence of the CAPM using unconditional distributional moments. Specifically, β_{i,R^w}, the unconditional covariance of returns, is constant. Unlike IID random variables, the conditional and unconditional moments of asset returns likely differ. Conditional moments are exepctations, variances, and covariances conditional on the available information set, e.g. $E_t(R_{t+1}) = E(R_{t+1}|I_t)$. The CAPM can hold conditionally, meaning be true in the cross-section for any time t, but fail unconditionally.

To see why, I derive the conditional CAPM. Begin with the fundamental asset pricing equation from an agent's two period consumption problem (note, this isn't a mere two-period model, the result follows for two arbitrary periods of a multiperiod model):

$$p_t = E_t\left[\beta\frac{u'(C_{t+1})}{u'(C_t)}x_{t+1}\right]$$

Equivalently:

$$p_t = E\left[\beta\frac{u'(C_{t+1})}{u'(C_t)}x_{t+1}\middle| I_t\right]$$

Let the SDF and gross return be, respectively:

$$m_{t+1} \equiv \beta\frac{u'(C_{t+1})}{u'(C_t)}$$
$$R_{t+1} \equiv \frac{x_{t+1}}{p_t}$$

Using the covariance decomposition $E(XY) = E(X)E(Y) + \mathrm{Cov}(X,Y)$:

$$1 = E_t(m_{t+1}R_{t+1})$$
$$1 = E_t(m_{t+1})E_t(R_{t+1}) + \mathrm{Cov}_t(m_{t+1}, R_{t+1})$$

Let the risk-free asset have a normalized price of 1 and gross return R^f:

$$1 = E_t\left[m_{t+1}R^f\right]$$
$$R^f = \frac{1}{E_t(m_{t+1})}$$

The CAPM assumes that the SDF is linear in the return on a total wealth portfolio:

$$m_{t+1} = a_t + b_t R^w_{t+1}$$

where a_t and b_t are free parameters. Using the Euler equation to price the risk free asset and wealth portfolio, I solve for a_t and b_t to make the time dependence explicit. Add a time subscript to the risk free rate to make it obvious that the risk-free return is part of the information set at time t:

$$1 = E_t(m_{t+1}R_{t+1})$$
$$1 = E_t(m_{t+1}R^f_t)$$
$$= R^f_t E_t(m_{t+1})$$
$$1 = E_t(m_{t+1}R^W_{t+1})$$

Obtain an expression for a_t:

$$1 = E_t(m_{t+1}R^f_t)$$
$$\frac{1}{R^f_t} = E_t(a_t + b_t R^W_{t+1})$$
$$a_t = \frac{1}{R^f_t} - b_t E_t(R^W_{t+1})$$

Now solve for b_t:

$$1 = E_t(m_{t+1})E_t(R^W_{t+1}) + \mathrm{Cov}_t(m_{t+1}, R^W_{t+1})$$
$$1 = \frac{1}{R^f_t}E_t(R^W_{t+1}) + \mathrm{Cov}_t(a_t + b_t R^W_{t+1}, R^W_{t+1})$$
$$1 - \frac{E_t(R^W_{t+1})}{R^f_t} = b_t \mathrm{Var}_t(R^W_{t+1})$$
$$b_t = \frac{R^f_t - E_t(R^W_{t+1})}{R^f_t \mathrm{Var}_t(R^W_{t+1})}$$
$$= \frac{E_t(R^W_{t+1}) - R^f_t}{-R^f_t \mathrm{Var}_t(R^W_{t+1})}$$

Derive the CAPM, using the assumption that the SDF is linear in the total wealth portfolio's return:

$$
\begin{aligned}
1 &= E_t(m_{t+1}R_{t+1}) \\
1 &= E_t(m_{t+1})E_t(R_{t+1}) + \mathrm{Cov}_t(m_{t+1}, R_{t+1}) \\
1 &= \frac{E_t(R_{t+1})}{R_t^f} + \mathrm{Cov}_t(m_{t+1}, R_{t+1}) \\
E_t(R_{t+1}) - R_t^f &= -R_t^f \mathrm{Cov}_t(m_{t+1}, R_{t+1}) \\
E_t(R_{t+1}) - R_t^f &= -R_t^f \mathrm{Cov}_t(a_t + b_t R_{t+1}^W, R_{t+1}) \\
&= -R_t^f \frac{E_t(R_{t+1}^W) - R_t^f}{-R_t^f \mathrm{Var}_t(R_{t+1}^W)} \mathrm{Cov}_t(R_{t+1}^W, R_{t+1}) \\
&= \frac{\mathrm{Cov}_t(R_{t+1}^W, R_{t+1})}{\mathrm{Var}_t(R_{t+1}^W)} (E_t(R_{t+1}^W) - R_t^f) \\
&= \beta_t \left[E_t(R_{t+1}^W) - R_t^f \right]
\end{aligned}
$$

Take unconditional expectations, using the Law of Iterated Expectations:

$$
\begin{aligned}
\lambda_t &\equiv E_t(R_{t+1}^W) - R_t^f \\
E(E_t(R_{t+1} - R_t^f)) &= E(\beta_t \lambda_t) \\
E(R_{t+1} - R_t^f) &= E(\beta_t)E(\lambda_t) + \mathrm{Cov}(\beta_t, \lambda_t)
\end{aligned}
$$

If the covariance between the market risk premium and the return's beta is zero - perhaps because conditional beta is constant, or the market risk premium moves in proportion to the market factor's conditional variance, etc. - then the simplified expected-return beta representation holds. The point is that the unconditional CAPM is really a specific case of a more general conditional model, and therefore it is entirely appropriate to test the CAPM as if beta is time-varying. Note, even with time varying beta, a time series regression test should produce an intercept that is statistically zero.

Let the time-varying CAPM model be:

$$
\begin{aligned}
R_t - Rf_t &= \alpha_t + \beta_t(Rm_t - Rf_t) + \epsilon_t, \epsilon_t \sim \mathrm{WN}(0, \sigma_e^2) \\
R_t^e &= \alpha_t + \beta_t Rm_t^e + \epsilon_t
\end{aligned}
$$

Where alpha and beta are time-varying. A stock's excess return in this 1 factor model is a function of its exposure to the market return, where the degree of co-movement (β_t) and returns net of market exposure (α_t) vary over time. Conditional variance is constant. My objective is to test recovery of the time-varying parameters from simulated data, and TV-CAPM is a useful vehicle for doing so.

Favorable asymptotics require large T, hence I use daily stock return, risk free rate, and market return data. 10 years of daily data produces T on the order of 2,000. Monthly, quarterly, or annual frequency is implausible for large T. For motivation, consider the last 10 years of AAPL and SP500 returns. I use the yield on the 3M UST bill as the risk free rate. For AAPL and the SP500, I use the closing daily, non-seasonally adjusted, price and level of the index, respectively. All returns are logarithmic.

$$
\begin{aligned}
R_t &= \ln(P_t^{\mathrm{AAPL}}) - \ln(P_{t-1}^{\mathrm{AAPL}}) \\
Rm_t &= \ln(P_t^{\mathrm{SP500}}) - \ln(P_{t-1}^{\mathrm{SP500}}) \\
Rf_t &= \frac{1}{365} \ln(1 + 0.01 \cdot Y_t)
\end{aligned}
$$

UST 1M daily yields, Y_t are annualized and reported in percentage points, so I scale appropriately to get a logarithmic daily risk free rate.

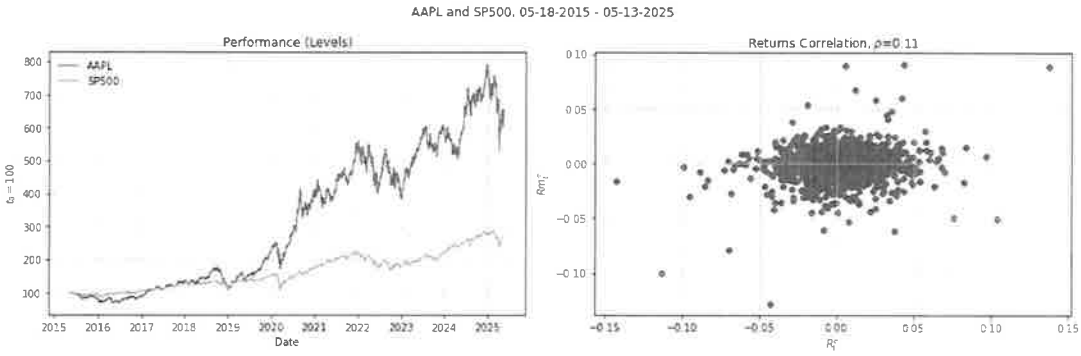
I estimate the model over the entire 5/15/2015 - 5/13/2025 sample, and then using 1-year rolling windows overlay the rolling estimates on top of the whole sample estimates with respective 95% CIs. I test the full sample model's parameter stability using a CUSUM test on the residuals.

```
#Can't use data pulls because of Citi firewall.
#Read in SP500 data from a CSV. Daily, index level, non-SA. Closing price.
sp500_df = pd.read_csv("C:/Users/RP64662/Downloads/SP500.csv")

#Format date column, compute log return
sp500_df = sp500_df.rename(columns={"observation_date":"Date","SP500":"Pm(t)"})
sp500_df["Date"] = pd.to_datetime(sp500_df["Date"])
sp500_df["Rm(t)"] = np.log(sp500_df["Pm(t)"]/sp500_df["Pm(t)"].shift(1))

#Read in UST 1-M daily yields from Fred. In %, non-SA.
ust_1m_df = pd.read_csv("C:/Users/RP64662/Downloads/DGS1MO.csv")
ust_1m_df = ust_1m_df.rename(columns={"observation_date":"Date","DGS1MO":"Y(t)"})
ust_1m_df["Date"] = pd.to_datetime(ust_1m_df["Date"])

#Compute daily RF rate, assuming continous compounding. Yields are annualized.
ust_1m_df["Rf(t)"] = (1/365)*np.log(1+ust_1m_df["Y(t)"]/100.0)

#AAPL
aapl_df = pd.read_csv("C:/Users/RP64662/Downloads/HistoricalData_1747318668456.csv")
aapl_df["Date"] = pd.to_datetime(aapl_df["Date"])
aapl_df["P(t)"] = aapl_df["Close/Last"].apply(lambda x: float(x.replace("$","")))
aapl_df["R(t)"] = np.log(aapl_df["P(t)"]/aapl_df["P(t)"].shift(1))

#Merge on Date, drop missing obs
capm_df = pd.merge(sp500_df[["Date","Rm(t)","Pm(t)"]], ust_1m_df[["Date","Rf(t)"]], on="Date", how="outer")
```

```
capm_df = capm_df.merge(aapl_df[["Date","R(t)","P(t)"]], on="Date", how="outer")
capm_df = capm_df.sort_values(by="Date").dropna().reset_index(drop=True)
capm_df = capm_df.set_index("Date")

#Compute price indices
capm_df["P(t)"] = 100.0*capm_df["P(t)"]/capm_df[capm_df.index==capm_df.index.min()]["P(t)"].values[0]
capm_df["Pm(t)"] = 100.0*capm_df["Pm(t)"]/capm_df[capm_df.index==capm_df.index.min()]["Pm(t)"].values[0]

#Compute excess returns
capm_df["R(t)-Rf(t)"] = capm_df["R(t)"] - capm_df["Rf(t)"]
capm_df["Rm(t)-Rf(t)"] = capm_df["Rm(t)"] - capm_df["Rf(t)"]

#Check co-movement
fig, axs = plt.subplots(nrows=1, ncols=2, figsize=(15,5))

#Cumulative performance
axs[0].plot(capm_df[["P(t)"]], label=r'AAPL')
axs[0].plot(capm_df[["Pm(t)"]], label=r'SP500')
axs[0].legend()
axs[0].set_title(f"Performance (Levels)")
axs[0].set_xlabel("Date")
axs[0].set_ylabel(r"$t_{0}=100$")
axs[0].grid()

#Return correlation
axs[1].scatter(capm_df[["R(t)-Rf(t)"]],capm_df[["Rm(t)-Rf(t)"]])
axs[1].set_title(f"Returns Correlation, "+r'$\rho$'+f"={np.round(np.corrcoef(capm_df['R(t)-Rf(t)'],capm_df['Rm(t)-Rf(t)']
axs[1].set_xlabel(r"$R^{e}_{t}$")
axs[1].set_ylabel(r"$Rm^{e}_{t}$")
axs[1].grid()
```

```
#Set title and display subplots
plt.suptitle(f"AAPL and SP500, {capm_df.index.min().strftime('%m-%d-%Y')} - {capm_df.index.max().strftime('%m-%d-%Y')}")
plt.tight_layout()
plt.show()

#Fit whole sample regression. Expect very low R2 because of daily noise.
Y_capm = capm_df["R(t)-Rf(t)"]
X_capm = sm.add_constant(capm_df["Rm(t)-Rf(t)"])
capm_model = sm.OLS(Y_capm, X_capm).fit(cov_type="HAC", cov_kwds={"maxlags":int(len(X_capm)**0.25)})
display(capm_model.summary())
```

```
#Cusum for OLS residual test
print("Performing CUSUM test on OLS residuals:")
try:
    #Perform test for param stability on residuals
    cusum_result = breaks_cusumolsresid(capm_model.resid, ddof=3)

    #Check if break present. H0: no structural change.
    if cusum_result[1] < 0.05:
        print("->Structural break in CAPM residuals present")
    else:
        print("->CAPM OLS parameters stable.")
except Exception as e:
    print(f"->Cusum Test Failure: {e}")

#Residual variance
print("Residual variance:",np.var(capm_model.resid,ddof=1))
```



OLS Regression Results

Dep. Variable:	R(t)-Rf(t)	**R-squared:**	0.012
Model:	OLS	**Adj. R-squared:**	0.012
Method:	Least Squares	**F-statistic:**	6.073
Date:	Sat, 21 Jun 2025	**Prob (F-statistic):**	0.0138

Time:	08:50:57	**Log-Likelihood:**	6181.4
No. Observations:	2401	**AIC:**	-1.236e+04
Df Residuals:	2399	**BIC:**	-1.235e+04
Df Model:	1		
Covariance Type:	HAC		

	coef	std err	z	P>|z|	[0.025	0.975]
const	-0.0010	0.000	-2.521	0.012	-0.002	-0.000
Rm(t)-Rf(t)	0.1736	0.070	2.464	0.014	0.036	0.312

Omnibus:	298.902	**Durbin-Watson:**	1.923
Prob(Omnibus):	0.000	**Jarque-Bera (JB):**	3078.332
Skew:	0.125	**Prob(JB):**	0.00
Kurtosis:	8.541	**Cond. No.**	85.6

Notes:
[1] Standard Errors are heteroscedasticity and autocorrelation robust (HAC) using 7 lags and without small sample correction

```
Performing CUSUM test on OLS residuals:
->CAPM OLS parameters stable.
Residual variance: 0.0003400419317060362
```

Full sample regression is quite low, at 1.2%. This is to be expected given the noise in daily return data. Typically the CAPM is estimated using monthly or annual returns, but daily data is desirable for testing purposes. Full sample (time-invariant) α and β estimates are significant at a 5% level using Newey-West standard errors. α is effectively zero and β is 17.4%.

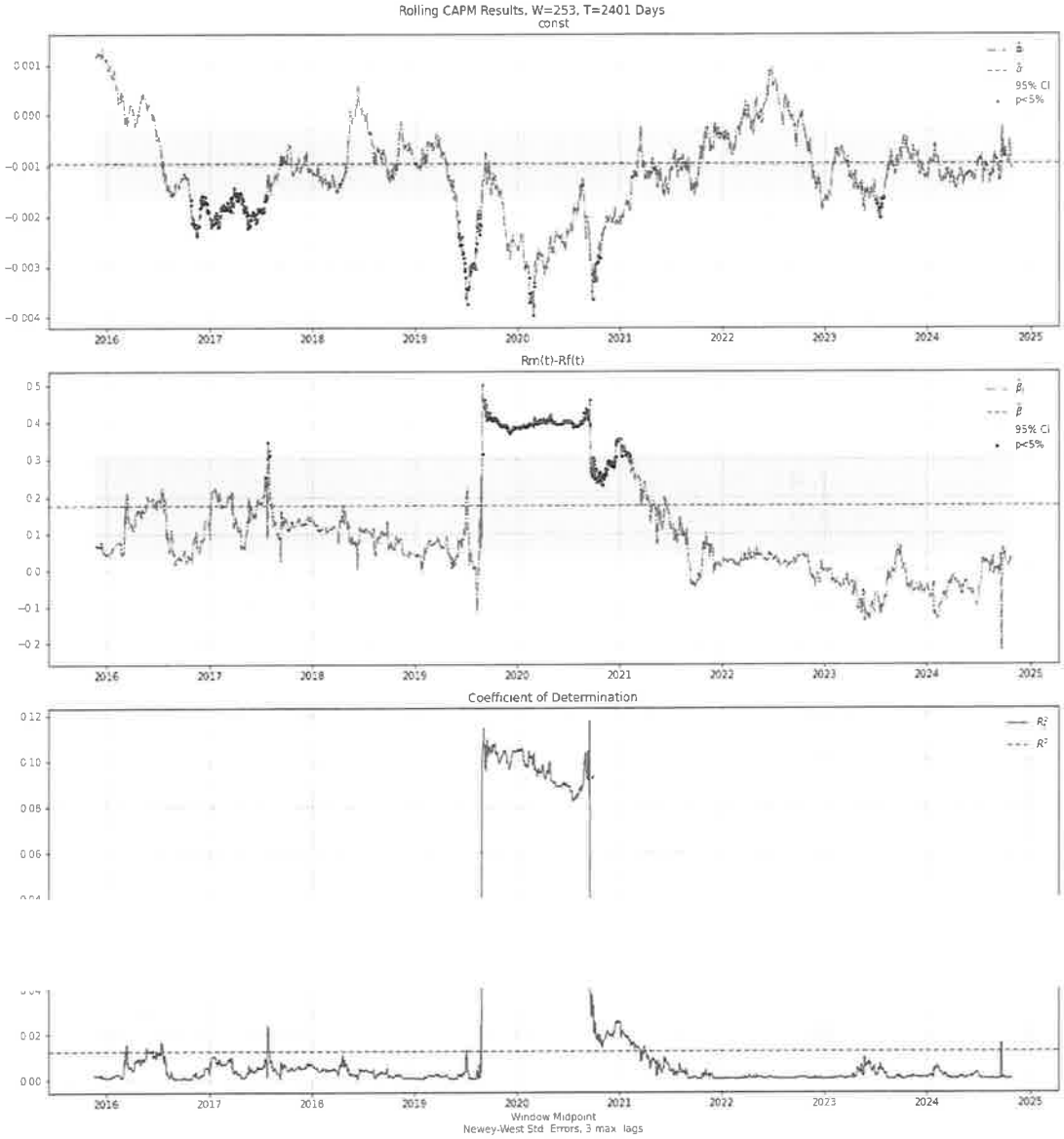
```
#Rolling, using W+1 day windows
rolling_results = []
w=252+1

#Roll through data, regress y on x with a constant, retain param est, R2, SEs, p-value
for i in range(0,len(Y_capm)-w+1):

    #Subset data




    Y_w = Y_capm.iloc[i:i+w]
    X_w = X_capm.iloc[i:i+w]

    #Store index range (inclusive, exclusive)
    idx_min, idx_max = Y_w.index.min(), Y_w.index.max()
    idx_mid = list(Y_w.index)[(len(Y_w) -1)//2]

    #Regress
    nw_lag = int(len(X_w)**0.25)
    w_model = sm.OLS(Y_w, X_w).fit(cov_type="HAC", cov_kwds={"maxlags":nw_lag})

    #CI for beta 0
    ci_b0_u = w_model.conf_int().T["const"][1]
    ci_b0_l = w_model.conf_int().T["const"][0]

    #CI for beta 1
    ci_b1_u = w_model.conf_int().T["Rm(t)-Rf(t)"][1]
    ci_b1_l = w_model.conf_int().T["Rm(t)-Rf(t)"][0]

    #Parameter estimates and p values
    b0_w = w_model.params["const"]
    p0_w = w_model.pvalues["const"]
    b1_w = w_model.params["Rm(t)-Rf(t)"]
    p1_w = w_model.pvalues["Rm(t)-Rf(t)"]

    #R2
    r2_w = w_model.rsquared

    #Store results
    w_results = {"Min.":idx_min, "Max.":idx_max, "CI.L.B0":ci_b0_l, "B0":b0_w, "CI.U.B0":ci_b0_u,"P(B0)":p0_w,
                                                 "CI.L.B1":ci_b1_l, "B1":b1_w, "CI.U.B1":ci_b1_u,"P(B1)":p1_w,
                 "R2":r2_w, "T":w_model.nobs, "Date":idx_mid}
    rolling_results.append(w_results)

#Store results as DF
rolling_results = pd.DataFrame(data = rolling_results)
print("\nRolling results:")
display(rolling_results)

#Plot
if True:
    rolling_results = rolling_results.set_index("Date")
    #Plot rolling estimates, CIs, p-values
    fig, axs = plt.subplots(nrows=3, ncols=1, figsize=(15,15))
```

```
    #Beta 0 and 1
    for b in [0,1]:

        if b==0:
            var_name = "const"
            plt_label = r"$\hat{\alpha}$"
        else:
            var_name = "Rm(t)-Rf(t)"
            plt_label = r"$\hat{\beta}$"

        #Plot rolling parameter estimate
        axs[b].plot(rolling_results[f"B{b}"], label=plt_label+r"$_{t}$", linestyle="dashdot")

        #Plot full sample estimate and it's 95% CI
        axs[b].axhline(capm_model.params[var_name], label=plt_label, linestyle="--")
        axs[b].fill_between(x=rolling_results.index,
                            y1=capm_model.conf_int().T[var_name][0],
                            y2=capm_model.conf_int().T[var_name][1], label="95% CI", alpha=0.2)

        #Overlay p-values
        axs[b].scatter(rolling_results.index,
                       np.where(rolling_results[f"P(B{b})"]<0.05, rolling_results[f"B{b}"], np.nan),
                       color="blue", label="p<5%", s=5)

        axs[b].set_title(w_model.model.exog_names[b])
        axs[b].grid()
        axs[b].legend()
        #axs[b].axhline(y=0, linestyle="--", color="black", alpha=0.5)

    #Plot rolling R2
    axs[2].plot(rolling_results["R2"], color="red", label=r'$R^{2}_{t}$')
    axs[2].axhline(capm_model.rsquared, color="red", linestyle="--", label=r'$R^{2}$')
    axs[2].set_title("Coefficient of Determination")
    axs[2].legend()
    axs[2].grid()

    #Set title, x label, and display
    plt.suptitle(f"Rolling CAPM Results, W={W}, T={len(Y_capm)} Days")
    plt.xlabel(f"Window Midpoint\nNewey-West Std. Errors, {nw_lag} max. lags")
    plt.tight_layout()
    plt.show()
```

```
Rolling results:
```

	Min.	Max.	CI.L.B0	B0	CI.U.B0	P(B0)	CI.L.B1	B1	CI.U.B1	P(B1)	R2	T	Date
0	2015-05-18	2016-06-03	-0.000981	0.001168	0.003318	0.286771	-0.199647	0.067993	0.335633	0.618540	0.001591	253.0	2015-11-20
1	2015-05-19	2016-06-06	-0.001003	0.001148	0.003298	0.295445	-0.200464	0.067018	0.334500	0.623376	0.001546	253.0	2015-11-23
2	2015-05-20	2016-06-07	-0.001000	0.001151	0.003301	0.294315	-0.200508	0.066979	0.334466	0.623584	0.001544	253.0	2015-11-24
3	2015-05-21	2016-06-08	-0.000991	0.001161	0.003313	0.290150	-0.201812	0.065541	0.332894	0.630884	0.001480	253.0	2015-11-25
4	2015-05-22	2016-06-09	-0.000925	0.001230	0.003384	0.263326	-0.201253	0.065992	0.333238	0.628396	0.001501	253.0	2015-11-30
...	...	...	...	...	...	...	...	...	...	...	...	...	...
2144	2024-04-16	2025-05-07	-0.003337	-0.000656	0.002024	0.631272	-0.401264	0.030988	0.463239	0.888258	0.000331	253.0	2024-10-22
2145	2024-04-17	2025-05-08	-0.003395	-0.000710	0.001975	0.604113	-0.401642	0.030929	0.463501	0.888549	0.000330	253.0	2024-10-23
2146	2024-04-18	2025-05-09	-0.003707	-0.000972	0.001763	0.486168	-0.399751	0.034206	0.468162	0.877223	0.000390	253.0	2024-10-24
2147	2024-04-19	2025-05-12	-0.003829	-0.001060	0.001709	0.453064	-0.399159	0.026589	0.452337	0.902578	0.000242	253.0	2024-10-25
2148	2024-04-22	2025-05-13	-0.003811	-0.001030	0.001750	0.467747	-0.400575	0.026192	0.452960	0.904253	0.000235	253.0	2024-10-28

2149 rows × 13 columns



Rolling estimates using a 1 year window are mostly within the 95% CI of the whole sample estimates, aside from the 2020 - 2021 period. The date indices in the plot above are the midpoints of the rolling windows. AAPL and SP500 return co-movement was much stronger during this period, with the market return explaining up to 12% of the variation in AAPLs return.

Solid dots indicate rolling estimates significant at a 5% level. Despite the CUSUM test showing whole sample regression estimates are stable, there are clear subperiods in which the noise in daily returns obscures any relationship between AAPL and SP500 returns and in which the model has considerably greater explanatory power, constituting evidence of time-varying parameters. α_t for AAPL is negative in 2017, for example.

Data Simulation

I proceed with data simulation. First I simulate random walks for α_t and β_t:

$$a_t = a_{t-1} + \eta_{1,t}$$
$$\beta_t = \beta_{t-1} + \eta_{2,t}$$

Drift is 0 for both parameters. The expected change is 0, not a constant, to represent parameters that gradually drift over time, randomly, instead of trending directionally or abruptly shifting. I assume that parameter white noise terms are uncorrelated with each other and that of process noise ϵ_t:

$$\begin{bmatrix} \epsilon_t \\ \eta_{1,t} \\ \eta_{2,t} \end{bmatrix} \sim \mathrm{WN}\left(\vec{0}, \Sigma_{\mathrm{WN}}\right)$$
$$\Sigma_{\mathrm{WN}} = \begin{bmatrix} \sigma_\epsilon^2 & 0 & 0 \\ 0 & \sigma_a^2 & 0 \\ 0 & 0 & \sigma_b^2 \end{bmatrix}$$

I use $n = 1,000$ simulations of length $T = 2500$ with 20% burn in, and the rolling OLS intercept and slope estimates for simulation drift and variance (regress estimated $\hat{a}_t$ on estimated $\hat{a}_{t-1}$, $\hat{b}_t$ on $\hat{b}_{t-1}$, compute volatility of AR(1) regression residuals). The exercise provides population parameters for simulation and also validates the use of random walks for simulations.

Assume AR(1) estimation bias is neglibible.

$$\hat{\alpha}_t = a_0 + \phi_a \hat{\alpha}_{t-1} + \eta_{a,t}, \eta_{a,t} \sim \mathrm{WN}(0, \sigma_a^2)$$
$$\hat{\beta}_t = b_0 + \phi_b \hat{\beta}_{t-1} + \eta_{b,t}, \eta_{b,t} \sim \mathrm{WN}(0, \sigma_b^2)$$

```
#Using Rolling OLS results from Motivation, regresses B(t) on B(t-1) and a(t) on a(t-1). Ignore AR(1) bias for a moment.
a_rols_model = sm.OLS(rolling_results["B0"].values[1:], sm.add_constant(rolling_results["B0"].values[0:-1])).fit()
b_rols_model = sm.OLS(rolling_results["B1"].values[1:], sm.add_constant(rolling_results["B1"].values[0:-1])).fit()
summary_col(results = [a_rols_model, b_rols_model],
            stars=True,
            model_names = ["a(t)","b(t)"],
            info_dict={"T": lambda x: x.nobs, "Resid. Vol.": lambda x: np.std(x.resid, ddof=1)} )
```

	a(t)	b(t)
const	-0.0000***	0.0008*
	(0.0000)	(0.0005)
x1	0.9916***	0.9920***
	(0.0025)	(0.0027)
R-squared	0.9864	0.9840
R-squared Adj.	0.9864	0.9840
T	2148.0000	2148.0000
Resid. Vol.	0.0001	0.0177

Standard errors in parentheses.
* p<.1, ** p<.05, ***p<.01

Check correlation of residuals for WN assumption.

```
np.corrcoef(a_rols_model.resid, b_rols_model.resid)
```

```
array([[ 1.        , -0.18772834],
       [-0.18772834,  1.        ]])
```

For large T, there's no need to correct bias. Simulate.

```
#Set n, T, burn in %. Each row is a simulation of length T.
n = 1000
T = 2500
burn_in = 0.20

#Compute # of points to discard (burn in)
burn_T = math.ceil(T*burn_in)

#Set parameters for a(t) and b(t) RWD processes
#a(t) follows RW with no drift. Include drift terms for general use.
sigma_a = np.std(a_rols_model.resid, ddof=1)
mu_a = 0.0
sigma_b = np.std(b_rols_model.resid, ddof=1)
mu_b = 0.0

#Simulate RWs with drift, n sims of length T, burn in
e_at = np.random.normal(0, sigma_a, size=(n,T + burn_T - 1))
e_bt = np.random.normal(0, sigma_b, size=(n,T + burn_T - 1))

#Change in a(t) is mu + N(0, sigma_a**2)
at_chg = mu_a + e_at
bt_chg = mu_b + e_bt

#Initial state
at_init = np.full((n,1), 0.0)
bt_init = np.full((n,1), 0.0)

#Concatenate initial state with change, then take cumulative sum
at_sim = np.cumsum(np.concatenate((at_init, at_chg), axis=1), axis=1)[:, burn_T:]
bt_sim = np.cumsum(np.concatenate((bt_init, bt_chg), axis=1), axis=1)[:, burn_T:]

#Fit AR(1)s to verify. Select random row, reg a(t) on a(t-1) and b(t) on b(t-1), compare params and std of resids
i = random.choice(range(n))
test_at = sm.OLS(at_sim[i,1:],sm.add_constant(at_sim[i,0:-1])).fit()
test_bt = sm.OLS(bt_sim[i,1:],sm.add_constant(bt_sim[i,0:-1])).fit()
print("Compare theoretical to simulated parameters:")
display(pd.DataFrame(data = [list(test_at.params)+ [np.std(test_at.resid, ddof=1)],
                             [mu_a, 1.0, sigma_a],
                             list(test_bt.params)+ [np.std(test_bt.resid, ddof=1)],
```

```
                [mu_b, 1.0, sigma_b]], columns=["Mu","Phi","Sigma"],
        index=["a(t) sim.", "a(t)", "b(t) sim.", "b(t)"]).round(3))
```

```
Compare theoretical to simulated parameters:
```

	Mu	Phi	Sigma
a(t) sim.	0.0	0.998	0.000
a(t)	0.0	1.000	0.000
b(t) sim.	0.0	0.999	0.017
b(t)	0.0	1.000	0.018

Sample moments of the simulated paths are reasonably close to the population values. To simulate market returns, I assume the SP500 index follows a geometric Brownian motion, and calibrate moments with historical SP500 return data. The parameters follow discrete time AR(1) processes, but I draw the covariate market return from a continuous time stochastic process for convenience. Simulated covariates are exogenous for the purposes of testing a model's ability to recover time-varying parameters.

A GBM assumes the SP500 index is log normally distributed, such that the log return on the index is normally distributed. Using the property of Brownian motions that increments are normally distributed ($B_t - B_s\ N(0, \sqrt{t-s})$), it is straightforward to simulate returns:

$$
\begin{aligned}
dS_t &= uS_t dt + \sigma S_t dB_t^P \\
S_t &= S_0 e^{(\mu - 0.5\sigma^2)t + \sigma B_t} \\
\ln\left(\frac{S_t}{S_0}\right) &= (\mu - 0.5\sigma^2)t + \sigma B_t \\
R_t &\equiv \ln(S_t/S_{t-1}) \\
&= (\mu - 0.5\sigma^2) + \sigma W, W \sim N(0,1)
\end{aligned}
$$

Lo and Mackinlay (The Econometrics of Financial Markets, 1996. pp.363.) derive the MLE estimators for the drift and diffusion terms. Letting $h = 1$ for intervals of length 1, and using their notation:

$$
\begin{aligned}
a &\equiv \mu - 0.5\sigma^2 \\
\hat{a}_{MLE} &= \frac{1}{T}\sum_{t=1}^{T} R_t \\
\hat{\sigma}^2_{MLE} &= \frac{1}{T}\sum_{t=1}^{T}(R_t - \hat{a}_{MLE})^2
\end{aligned}
$$

However, the parameter process is discrete time. Start with the general form of SDE, for drift $a(X_t, t)$ and diffusion $b(X_t, t)$:

$$
\begin{aligned}
dX_t &= a(X_t, t)dt + b(X_t, t)dW_t \\
\Delta W_t &= W_{t+\Delta t} - W_t \\
&\sim N(0, \sqrt{\Delta t}^2) \\
\Delta W_t &= \sqrt{\Delta t} Z \\
Z &\sim N(0,1)
\end{aligned}
$$

Integrate the SDE:

$$
\begin{aligned}
\int_t^{t+\Delta t} dX_s &= \int_t^{t+\Delta t} a(X_s, s)ds + \int_t^{t+\Delta t} b(X_s, s)dW_s \\
X_{t+\Delta t} - X_t &= \int_t^{t+\Delta t} a(X_s, s)ds + \int_t^{t+\Delta t} b(X_s, s)dW_s
\end{aligned}
$$

Approximate the integral by assuming that for a minute increment Δt, the drift and diffusion terms are approximately constant:

$$
\begin{aligned}
\int_t^{t+\Delta t} a(X_s, s)ds &\approx a(X_t, t)\int_t^{t+\Delta t} ds \\
&= a(X_t, t)\Delta t \\
\int_t^{t+\Delta t} b(X_s, s)dW_s &\approx b(X_t, t)\int_t^{t+\Delta t} dW_s \\
&= b(X_t, t)(W_{t+\Delta t} - W_t) \\
&= b(X_t, t)\Delta W_t
\end{aligned}
$$

Substitute and rearrange:

$$
\begin{aligned}
X_{t+\Delta t} - X_t &\approx a(X_t, t)\Delta t + b(X_t, t)\Delta W_t \\
&= X_t + a(X_t, t)\Delta t + b(X_t, t)\Delta W_t
\end{aligned}
$$

For a GBM,

$$
\begin{aligned}
dS_t &= \mu S_t dt + \sigma S_t dW_t^P \\
S_{t+\Delta t} &\approx S_t + \mu S_t \Delta t + \sigma S_t \Delta W_t^P \\
&= S_t + \mu S_t \Delta t + \sigma S_t \sqrt{\Delta t} Z
\end{aligned}
$$

I estimate the parameters of the GBM using 10 years of SP500 data, and simulate from the process, and plot a few simulated paths (in levels) of the SP500 index to illustrate.

```
#MLE estimates for mu, sigma:
a = np.mean(sp500_df["Rm(t)"].dropna())
sigma_sq_hat = np.mean((sp500_df["Rm(t)"].dropna() - a)**2)
mu_hat = a + 0.5*sigma_sq_hat
print(sp500_df[["Date","Rm(t)"]].dropna().min(), sp500_df[["Date","Rm(t)"]].dropna().max(),
      len(sp500_df["Rm(t)"].dropna()))
print("Mu hat:",mu_hat, "Sigma Sq Hat",sigma_sq_hat)
#GBM simulations. 2500 time steps.
dt = 1.0
drift = (mu_hat - 0.5 * sigma_sq_hat)*dt
diffusion = np.sqrt(sigma_sq_hat)*np.sqrt(dt)

#Brownian motion increments are N(0, sqrt(dt))
bt_shocks = np.random.randn(n, T)
ln_ret = drift + diffusion*bt_shocks
cum_ln_ret = np.cumsum(ln_ret, axis=1)

#Exponentiate cum log returns, mutply by initial price. Add S0 as first col.
S0 = sp500_df["Pm(t)"].dropna().values[0]
spx_paths = np.concatenate( (np.full((n,1),S0) ,S0*np.exp(cum_ln_ret)  ), axis=1)

#Create DF of simulated paths and actual SP500 for plotting
plot_spx = pd.DataFrame(spx_paths[0:5,:]).T
plot_spx.index = sp500_df["Date"].dropna().values[0:T+1]
plot_spx["SP500 Actual"] = sp500_df["Pm(t)"].dropna().values[0:T+1]

#Plot 5 sample paths and actual
fig, ax = plt.subplots(figsize=(15,10))
ax.plot(plot_spx["SP500 Actual"], label="SP500 Actual", linestyle="-", color="black" )
plot_spx.drop(columns=["SP500 Actual"]).plot(alpha=0.5, linestyle="--", ax = ax)

#Set title, axis labels, use legend and grid
ax.set_title(f"GBM Simulations: SP500, {min(plot_spx.index).strftime('%m-%d-%Y')} - {max(plot_spx.index).strftime('%m-%d-
ax.set_xlabel("Date")
ax.set_ylabel(r"$P_{t}$")
ax.legend()
ax.grid()
plt.show()
```

```
Date     2015-05-18 00:00:00
Rm(t)              -0.127652
```

```
dtype: object Date     2025-05-14 00:00:00
Rm(t)                0.090895
dtype: object 2420
Mu hat: 0.0005326406784490215 Sigma Sq Hat 0.00013595490219255114
```

Sample paths appear reasonable. Finally, I combine the random walks for α_t, β_t with GBM simulations for SP500 returns and randomly resampled risk-free rates to construct simulated returns. Concisely, having simulated α_t, β_t and Rm_t, and drawing Rf_t using nonparametric resampling, I compute $R_t - Rf_t$.

```
#Set variance for e(t) using whole sample regression residual volatility
sigma_e = capm_model.resid.std(ddof=1)
```

```
#Compute log returns on simulated SPX, subtract RF rate
rm_paths = np.log(spx_paths[:,1:]/spx_paths[:,:-1])

#Randomly sample from observed RF
sim_rf = np.random.choice(ust_1m_df["Rf(t)"].dropna().values, size=T, replace=True)
rm_rf_paths = rm_paths - sim_rf

#Generate simulated excess returns R(t) - R(f) using R(t) - Rf = a(t) + Beta(t)*(Rm(t) - Rf(t)) + e(t)
re_sim = at_sim + np.multiply(bt_sim, rm_rf_paths) + np.random.normal(0, sigma_e, size=(n,T) )

#Check appropriate shape
try:
    assert(re_sim.shape==(n,T))
    assert(rm_rf_paths.shape==(n,T))
except AssertionError as e:
    print(f"{e}: simulated matrix not dimension {(n,T)}")
```

3a. Testing Procedure

The testing procedure to recover simulated time-varying parameters α_t, β_t is as follows.

-> For each estimation method:

---> For each simulation (i):

-----> Estimate α_t^i, β_t^i using $R_t^i - Rf_t^i$ as a dependent variable and $Rm_t^i - Rf_t^i$ as the independent variable.

-----> Compute error measures over t=1...T (in practice, using vectors) for α_t^i, β_t^i and for $R_t^i - Rf_t^i$.

Compare each method's distributions of errors computed over i simulations. The goal of the exercise is to identify which method best both recovers time-varying parameters and models the overall time-varying relationship.

3b. Time-Varying Parameter Estimation

1) State Space Model (KF)

Recast the model:

$$
\begin{aligned}
R_t - Rf_t &= \alpha_t + \beta_t(Rm_t - Rf_t) + \epsilon_t \\
R_t^e &= \alpha_t + \beta_t Rm_t^e + \epsilon_t \\
a_t &= a_{t-1} + \eta_{1,t} \\
b_t &= b_{t-1} + \eta_{2,t}
\end{aligned}
$$

in state space form. Define the state and measurement equations as:

$$
\begin{aligned}
X_t &= \phi X_{t-1} + W_t \\
Y_t &= A_t X_t + V_t
\end{aligned}
$$

respectively. Y_t is observed, A_t the observation matrix, X_t is the unobserved state variable, and W_t and V_t are noise terms:

$$
X_t \equiv \begin{bmatrix} \alpha_t \\ \beta_t \end{bmatrix}, \Phi = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, W_t = \begin{bmatrix} \eta_{1t} \\ \eta_{2t} \end{bmatrix}
$$

$$
Y_t = R_t^e, A_t = [1 \; Rm_t^e], V_t \equiv \epsilon_t
$$

Further, let:

$$
\begin{aligned}
W_t &\overset{\text{iid}}{\sim} N(0, Q) \\
Q &= \begin{bmatrix} \sigma_a^2 & 0 \\ 0 & \sigma_b^2 \end{bmatrix} \\
V_t &\overset{\text{iid}}{\sim} N(0, R) \\
R &= \sigma_e^2
\end{aligned}
$$

The Kalman Filter is a projection: $X_t^t \equiv E(X_t|Y_1 \ldots Y_t)$, with corresponding conditional error variance $P_t^t = E[(X_t - X_t^t)(X_t - X_t^t)'|Y_1 \ldots Y_t)$. Set two initial conditions:

$$
\begin{aligned}
X_0^0 &= x_0^0 \\
P_0^0 &= \Sigma_0
\end{aligned}
$$

and for $t = 1 \ldots T$, iteratively compute the state update equations, Kalman filter, MSE, and Kalman gain, respectively:

$$
\begin{aligned}
X_t^{t-1} &= \Phi X_{t-1}^{t-1} \\
P_t^{t-1} &= \Phi P_{t-1}^{t-1} \Phi' + Q \\
X_t^t &= X_t^{t-1} + K_t(Y_t - A_t X_t^{t-1}) \\
P_t^t &= [I - K_t A_t] P_t^{t-1} \\
K_t &= P_t^{t-1} A_t' [A_t P_t^{t-1} A_t' + R]^{-1}
\end{aligned}
$$

The Kalman filter has one key advantage: it is straightforward to updated the state variable from X_{t-1}^{t-1} to X_t^t given a new observation Y_t, without cycling through all prior observed $Y_1 \ldots Y_{t-1}$. Once formulated as a state space model, estimation is easy using MLE.

```
def kalman_filter_capm(y, x, sigma_a = 1.0, sigma_b = 1.0, sigma_e = 1.0, T = None, n_states=2,
                       init_state_mean = None, init_state_covariance = None):
    """
    A function to use the Kalman Filter to estimate time-varying a(t) and b(t) in the CAPM:
    R(t) - Rf(t) = a(t) + Beta(t)*(Rm(t)-Rf(t)) + e(t)
    a(t) = a(t-1) + eta1(t)
    b(t) = b(t-1) + eta2(t)
    Args:
        y: pd.Series, a vector of excess stock returns
        x: pd.Series, a vector of excess market returns
        sigma_a: float, volatility of a(t)'s RW WN term
        sigma_b: float, volatility of b(t)'s RW WN term
        sigma_e: float, volatility of process WN term
        T: int, number of observation
        n_states: int, the number of time varying parameters (alpha and beta)
        init_state_mean: np.array, an initial guess for state vector
        init_state_covariance: np.array, initial estiamte of state noise error
    Returns:
        kf_results: dict, a dictionary containing parameter estimates, CIs, p-values
    """

    #Check for valid inputs
    if len(y)!=len(x):
        raise ValueError("Input vectors not same length.")
    if not ((sigma_a > 0.0) & (sigma_b > 0.0) & (sigma_e > 0.0)):
        raise ValueError("Negative variance specified.")

    #Set T if not passed in
    if T is None:
        T = len(y)

    """




    State Space Matrices
    """
    #Define transition matrix Phi
    Phi = np.identity(n_states)

    #Define parameter process noise
    Q = np.diag([sigma_a**2, sigma_b**2])

    #Define observation process noise
    R = np.array([[sigma_e**2]])

    #Initalize arrays to hold results
    #Initial parameter guess
    if init_state_mean is None:
        state_means = np.zeros((T, n_states))
    else:
        state_means = init_state_mean

    #Initial state covariance
    if init_state_covariance is None:
        state_covariances = np.zeros((T, n_states, n_states))
    else:
        state_covariances = init_state_covariance

    #Residuals and kalman gains
    residuals = np.zeros(T)
    residual_covariances = np.zeros(T)
    kalman_gains = np.zeros((T,n_states))

    #Set initial values for theta and P
    theta_hat = np.array([[0.0],[0.0]])
    P = np.identity(n_states)

    #Loop for filter updates
    for t in range(T):

        #Prediction
        theta_hat_prior = Phi @ theta_hat
        P_prior = Phi @ P @ Phi.T + Q

        #Update
        A_t = np.array([[1, x.iloc[t]]])

        #Measurement innovation (residual)
        y_t = y.iloc[t]
```

```
        residual = y_t - A_t @ theta_hat_prior

        #Residual covariance
        residual_covariance = A_t @ P_prior @ A_t.T + R

        #Kalman gain
        kalman_gain = P_prior @ A_t.T * (1.0/residual_covariance[0,0])

        #Update state estimate
        theta_hat_post = theta_hat_prior + kalman_gain * residual

        #Update error covariance
        P_post = (np.eye(n_states) - kalman_gain @ A_t) @ P_prior

        #Store results
        state_means[t,:] = theta_hat_post.flatten()
        state_covariances[t,:,:] = P_post
        residuals[t] = residual
        residual_covariances[t] = residual_covariance[0,0]
        kalman_gains[t,:] = kalman_gain.flatten()

        #Set next iter estimates
        theta_hat = theta_hat_post
        P = P_post


    #Extract params, CIs, t stats
    kf_alpha = state_means[:,0]
    kf_beta = state_means[:,1]

    #Variacnes, SEs
    alpha_variance = state_covariances[:,0,0]
    beta_variance = state_covariances[:, 1, 1]
    alpha_std_err = np.sqrt(np.maximum(alpha_variance, 1e-10))
    beta_std_err = np.sqrt(np.maximum(beta_variance, 1e-10))

    #95% CI
    z_score_95 = norm.ppf(0.975)
    alpha_ci_l = kf_alpha - z_score_95*alpha_std_err
    alpha_ci_u = kf_alpha + z_score_95*alpha_std_err
    beta_ci_l = kf_beta - z_score_95*beta_std_err
    beta_ci_u = kf_beta + z_score_95*beta_std_err

    #T stats
    alpha_t_stat = kf_alpha/alpha_std_err




    beta_t_stat = kf_beta/beta_std_err

    #P val
    alpha_p = 2*norm.sf(np.abs(alpha_t_stat))
    beta_p = 2*norm.sf(np.abs(beta_t_stat))

    #Return dictionary of results
    kf_results= {"a(t)":kf_alpha,
                 "b(t)":kf_beta,
                 "CI a(t)":(alpha_ci_l, alpha_ci_u),
                 "CI b(t)":(beta_ci_l, beta_ci_u),
                 "p a(t)":alpha_p,
                 "p b(t)":beta_p}
    return kf_results

#Sample function call
#print("Test KF function output:")
#kalman_filter_capm(pd.Series(re_sim[0,:]), pd.Series(rm_rf_paths[0,:]))
```

2) Kernel Regression

Rolling OLS, algorithmically, entails:

-> For t in 1....T:

---> Take a subset of size $W+1$ of the dependent variable and covariate:

---> $Y_t = [y_{t-\frac{w}{2}} \dots y_t \dots y_{t+\frac{w}{2}}]'$

---> $X_t = [x_{t-\frac{w}{2}} \dots x_t \dots x_{t+\frac{w}{2}}]'$

---> Solve for β: $\min_\beta ||(Y_t - X_t\beta)||_2^2$

-> Collect $\hat{\beta}_t$, and associated measures of statistical significance and goodness of fit

Rolling OLS at each t is equivalent to performing weighted least squares with a boxcar kernel. Observations outside the kernel's window get a weight of 0 and observations inside are equal weighted.

I outline kernel regression broadly. For kernel $K(\cdot)$ and bandwidth h, solve the WLS problem:

$$\min_{\beta(t)} \sum_{i=1}^{T} (y_i - x_i'\beta(t))^2 K(\frac{t-i}{h})$$

A valid Kernel function is non-negative, symmetric, and integrates to 1 over its support. Fairly similar to a probability density function aside from the symmetry requirement:

$$1. K(x) \geq 0 \; \forall x$$
$$2. \int_{-\infty}^{\infty} K(u)\mathrm{du} = 1$$
$$3. K(x) = K(-x) \; \forall x$$

There are many choices for a kernel function. Rolling OLS (centered) is kernel regression but with a boxcar (aka uniform, rectangular) kernel:

$$K(x) = \mathbf{1}(|x| \leq c) \cdot \frac{1}{2c}$$

for c=1. Generic formulation above. Kernel functions $K\left[(t-i)h^{-1}\right]$ can assign greater weight to observations closer to a target and lower weight to more distant observations. The smoothing parameter h is the bandwidth. Larger h results in smoother estimates, smaller h results in noisy estimates. To see why, consider the Gaussian kernel:

$$K(x) = \frac{1}{\sqrt{2\pi}} e^{\frac{-x^2}{2}}$$

the support of which is the real line, so each observation's weight is strictly positive. All observations get a non-zero weight, but the weight rapidly decays for large $|x|$. As $h->\infty$, $K(\cdot)->(2\pi)^{-0.5}$ and the optimization problem collapses to OLS on the entire sample. Parameter estimates are time invariant (flat). As $h->0$, holding $t-i$ constant, the weights go to 0 as ratio (t-i)/h increases, and more weight on closest observations. The result is noisier parameter estimates on account of just using local data.

Uniform and gaussian kernels are two extremes. Gaussian has infinite support, uniform does not. Weight goes from constant to 0 at boundaries, which can induce more variance in rolling estimates (sharp transition). An infinite support uses all data, and so is more computationally costly, whereas a compact support is more efficient.

Use CV to calibrate bandwidth h or a heuristic. There is no window size. Bandwidth is key. CV technique should incorproate AR structure of parameters. Can use:

- one period ahead prediction for rolling window: vary W, for each candidate W, perform rolling OLS, use IS fit for one period ahead forecast, compute forecast error, average over obs, select W minimzing avg forecasting error). If parameters are constant, forecast error should avg to 0.
- typical LOOCV. For each t, omit the obs at t, and fit kernel reg. using a candidate h value at t-k...t-1,t+1...t+k. At each of those t-i values, compute forecast error for omitted t. Average, or take weighted average using kernel. Take avg forecast error across t, repeat for candidate h's.
- for each candidate h, perform kernel reg as normally done and obtain time varying estimates of a(t) and b(t). Fit AR(1)s to a(t) and b(t) - regress a(t) on a(t-1) and b(t) on b(t-1) - and compute SSR. CV h's score is SSR for a plus SSR for b, choose the score minimizing h. This requires some work, it seems like picking the h that produces the best parameter recovery of a(t) and b(t) instead of best recovery of y(t), but the goal is parameter recovery, so it is too much like selecting on OOS minimzing error. An improvement is performing LOO first, so for each t, omit obs. at t, take kernel estimates of a(t) and b(t), maybe using avg. of est. at a(t-1) and a(t+1), then do AR(1) SSR procedure. This stops CV from picking a tiny bandwidth that tracks noise e(t) in the process for y(t).
- Default methods tie h to sample size T, e.g. $h = T^{0.5}, h = T^{0.4}$.

The procedure requires performing a WLS regression at each time observation. Let:

- W_t is a diagonal matrix with time varying elements $K(\frac{t-1}{h}), K(\frac{t-2}{h}), \ldots, K(\frac{t-(T-1)}{h}), K(\frac{t-T}{h})$ such that $W_t = W_t'$. X the $Tx2$ design matrix, and Y a $TX1$ vector of responses:

$$W_t = \begin{bmatrix} K(\frac{t-1}{h}) & \cdots & & 0 \\ & K(\frac{t-2}{h}) & & \\ \vdots & & \ddots & \vdots \\ & & K(\frac{t-(T-1)}{h}) & \\ 0 & & & K(\frac{t-T}{h}) \end{bmatrix}_{T \times T}$$

$$X = \begin{bmatrix} 1 & x_1 \\ \vdots & \\ 1 & x_T \end{bmatrix}_{T \times 2}, Y = \begin{bmatrix} y_1 \\ \vdots \\ y_T \end{bmatrix}_{T \times 1}, \beta_t = \begin{bmatrix} \beta_{0,t} \\ \beta_{1,t} \end{bmatrix}_{2 \times 1}$$

$$Y = X\beta_t + \epsilon, \epsilon \sim N(\vec{0}, \sigma_\epsilon^2 I_{TxT})$$

Note that the covariate and response matrices are time-invariant. For each t=1...T, compute parameter estimates, standard errors, confidence intervals, and p-values using:

$$
\begin{aligned}
\hat{\beta}_t &= (X'W_tX)^{-1}(X'W_tY) \\
\mathrm{Var}\left(\hat{\beta}_t|X\right) &= (X'W_tX)^{-1}\mathrm{Var}\left(X'W_tX\beta_t + X'W_t\epsilon|X\right)(X'W_tX)^{-1} \\
&= (X'W_tX)^{-1}X'W_t\mathrm{Var}\left(\epsilon|X\right)W_t'X(X'W_tX)^{-1} \\
&= (X'W_tX)^{-1}X'W_t\sigma_\epsilon^2 IW_t'X(X'W_tX)^{-1} \\
&= \sigma_\epsilon^2(X'W_tX)^{-1}X'W_tW_t'X(X'W_tX)^{-1} \\
&= \sigma_\epsilon^2(X'W_tX)^{-1}X'W_t'W_tX(X'W_tX)^{-1}
\end{aligned}
$$

Unlike conventional WLS, where $\mathrm{Var}(\epsilon_t) = \sigma_\epsilon^2/w_t$ and the covariance matrix of $\hat{\beta}_{WLS} = \sigma^2(X'WX)^{-1}$, the interior term does not simplify. If $W_t = I$, the expression simplifies to the variance of the OLS estimator. Note, $X'WX$ is symmetric, and $(A')^{-1} = (A^{-1})'$ generally, so $((X'WX)')^{-1} = (X'WX)^{-1}$.

To estimate the variance σ_ϵ^2, I use the weighted sum of squared errors divided by the degrees of freedom, where I computed the weighted SSE using the time varying parameter estimates and weights. The estimate of σ_ϵ^2 is therefore also time-varying:

$$
\begin{aligned}
\hat{\sigma}_{e,t}^2 &= \frac{(Y - X\hat{\beta}_t)'W(Y - X\hat{\beta}_t)}{T-2-1} \\
\hat{\mathrm{Cov}}(\beta_t) &= \hat{\sigma}_{e,t}^2(X'W_tX)^{-1}X'W_t'W_tX(X'W_tX)^{-1}
\end{aligned}
$$

Standard errors are the square root of the diagonal elements of the estimated covariance matrix. I construct CIs using the form:

$$
\begin{aligned}
\hat{\beta}_{0,t} &\pm 1.96 \cdot \mathrm{SE}(\hat{\beta}_{0,t}) \\
\hat{\beta}_{1,t} &\pm 1.96 \cdot \mathrm{SE}(\hat{\beta}_{1,t})
\end{aligned}
$$

Given the large sample size, the estimate of the degrees of freedom, use of a 1.96 CV corresponding to a 95% CI, the absence of controls for heteroskedasticity and autocorrelation in ϵ, is tolerable.

With a time-varying parameter vector, β_t, kernel regression also produces time-varying predictions. Performing kernel regression at each t produces parameter estimates at each t, and therefore in sample predictions $\hat{Y}_t = X\hat{beta}_t$, MAPE over the entire sample, and R2 over the entire sample at each t. Prediction quality of $\hat{y}_t$ should be highest given the kernel places the most weight on the observation at t, so I compute point APE at t as well.

$$
\begin{aligned}
\mathrm{MAPE}_t &= |\frac{X\hat{\beta}_t - Y}{Y}| \\
\mathrm{APE}_t &= |\frac{x_t\hat{\beta}_t - y_t}{y_t}|
\end{aligned}
$$

T is so large that p-values are almost moot. It's feasible to use T - (p+1) for degrees of freedom in the estimate of error variance because T is so large. SEs are square root of diagonal elements of estimated var/cov matrix. Construct CIs using estimate $\pm$ 1.96 times SE. For p-value, use survival function of t dist.

```
#Kernel regression
def kernel_reg(Y, X, h=None, plot=False, kernel= lambda x:((2*np.pi)**-0.5)*np.exp(-0.5*(x**2)) ):
    '''
    Perform Kernel Regression.
    Args:
        Y: array like, vector of response observations
        X: array like, vector of covariate observations
        h, float: bandwidth for Gaussian kernel.
        plot, boolean: flag to generate plots
        kernel, function: kernel function. Defaults to Gaussian.
    Returns:
        kernel_results, pd.DataFrame: DF containing parameter estimates, CIs, p-values, R2, MAPE, point APE
    Default kernel bandwidth is T^0.5. Literature suggests T^0.5 or T^0.4.
    '''
    #Check for valid inputs
    if len(Y) != len(X):
        raise ValueError(f"Response vector Y length {len(Y)}, differs from covariate vector length {len(X)}.")

    #Reshape, prepend column of 1s to X
    Y = np.array(Y).reshape(len(Y),1)
    X = np.array(X).reshape(len(X),1)
    X = np.hstack((np.ones((len(X),1)), X))

    #Default behavior for kernel bandwidth
    if h is None:
        #Set BW = T^0.5
        h = len(Y)**0.5


    #Define index for kernel function
    i = np.arange(1,len(Y)+1)

    #WLS est. W is a diagonal matrix with weight for obs i, w_i, at position (i,i) and 0s elsewhere
```

```python
def beta_wls(X, W, Y): return np.linalg.inv(X.T @ W @ X) @ (X.T @ W @ Y)

#WLS SE and P-values. MSE is estimate of unknown variance sigma^2
def cov_wls(X, W, Y, mse):
    return mse*(np.linalg.inv(X.T @ W @ X) @ (X.T @ W.T @ W @ X) @ np.linalg.inv(X.T @ W @ X))

#per t, estimate beta, SE, p-val, R2, and point APE
kernel_results = []
for t in range(1, len(Y)+1):

    #compute weights
    w_t = kernel((t-i)/h)

    #W is a diagonal matrix with w_i at (i,i)
    W = np.diag(w_t)

    #Compute beta estimates
    try:
        beta = beta_wls(X, W, Y)
        b0, b1 = beta[0][0], beta[1][0]

        #Compute SSE, MSE, R2, MAPE, and point APE
        Y_pred = X @ beta
        sse = (Y - Y_pred).T @ W @ (Y - Y_pred)
        mse = sse/(len(Y) - 2 - 1)
        r2 = 1 - np.sum((Y - Y_pred)**2)/np.sum((Y - np.mean(Y))**2)
        mape = 100.0*np.mean(np.abs(Y - Y_pred)/np.abs(Y))
        t_ape = 100.0*np.abs(Y[t-1] - Y_pred[t-1])/np.abs(Y[t-1])[0]

        #Compute covariance of beta
        cov_beta = cov_wls(X,W, Y, mse)

        #Extract standard errors
        se_b0 = np.sqrt(cov_beta[0][0])
        se_b1 = np.sqrt(cov_beta[1][1])

        #Construct CIs. 5% two sided, using t dist w/ T-p-1 DF.
        ci_b0_l = b0 - 1.9615*se_b0
        ci_b0_u = b0 + 1.9615*se_b0
        ci_b1_l = b1 - 1.9615*se_b1
        ci_b1_u = b1 + 1.9615*se_b1

        #p-value. (beta_hat - 0) / se ~ t(n-p-1)
        p_b0 = stats.t.sf(np.abs(b0/se_b0), len(Y)-2-1)*2
        p_b1 = stats.t.sf(np.abs(b1/se_b1), len(Y)-2-1)*2




    except np.linalg.LinAlgError as l:
        """
        Error at last value of t using custom kernel, because only (t,t) element of
        #weight matrix is non-zero. All else are 0, so reg. tries using 1 obs w/intercept
        and covariate hence Linear Algebra error
        """
        print(f"Linear Algebra Error: {l} at t={t}.")

        #If so, set all values to non
        ci_b0_l, b0, ci_b0_u, p_b0 = np.nan, np.nan, np.nan, np.nan
        ci_b1_l, b1, ci_b1_u, p_b1 = np.nan, np.nan, np.nan, np.nan
        r2, t_ape, mape = np.nan, np.nan, np.nan

    #This segement executes regardless of error
    else:
        #Track results
        t_results = {"t":t,"CI.L.B0":ci_b0_l, "B0":b0, "CI.U.B0":ci_b0_u,"P(B0)":p_b0,
                     "CI.L.B1":ci_b1_l, "B1":b1, "CI.U.B1":ci_b1_u,"P(B1)":p_b1,
                 "R2":r2, "APE":t_ape, "MAPE":mape}
        kernel_results.append(t_results)

#Store as DF
kernel_results = pd.DataFrame.from_dict(kernel_results)

#Plot
if plot:
    #Plot kernel estimates, CIs, p-values
    fig, axs = plt.subplots(nrows=3, ncols=1, figsize=(15,10))

    #Beta 0 and 1
    for b in [0,1]:

        axs[b].plot(kernel_results.t, kernel_results[f"B{b}"], label=r"$\hat{\beta}$"+str(b))
        axs[b].fill_between(x=kernel_results.t,
                            y1=kernel_results[f"CI.L.B{b}"],
                            y2=kernel_results[f"CI.U.B{b}"], label="95% CI", alpha=0.2)

        #Overlay p-values
        axs[b].scatter(kernel_results.t,
                       np.where(kernel_results[f"P(B{b})"]<0.05, kernel_results[f"B{b}"], np.nan),
                       color="blue", label="p<5%", s=5)

        if b==0:
            axs[b].set_title("const")
        else:
```

```
            axs[b].set_title("x1")
        axs[b].grid()
        axs[b].legend()
        axs[b].axhline(y=0, linestyle="--", color="black", alpha=0.5)

    #Plot R2
    axs[2].plot(kernel_results.t, kernel_results.R2, color="red")
    axs[2].set_title(r"$R^{2}$")
    axs[2].grid()

    #Set title, x label, and display
    plt.suptitle(f"Kernel Regression Results, T={len(Y)}")
    plt.xlabel(f"t")
    plt.tight_layout()
    plt.show()

    #Separate plots for MAPE and APE
    fig, axs = plt.subplots(nrows=2, ncols=1, figsize=(15,10))
    axs[0].plot(kernel_results.t, kernel_results.MAPE, color="red")
    axs[0].set_title(r"$MAPE(t)$")
    axs[0].set_ylabel("%")
    axs[0].grid()
    axs[1].plot(kernel_results.t, kernel_results.APE, color="red", linestyle="--")
    axs[1].set_title(r"$APE(t)$")
    axs[1].set_ylabel("%")
    axs[1].grid()
    plt.suptitle(f"Kernel Regression Results, T={len(Y)}")
    plt.xlabel(f"t")
    plt.tight_layout()
    plt.show()

#Return dataframe
return kernel_results


#Sample function call
#print("Test Kernel Regression Output:")
#kernel_reg(pd.Series(re_sim[0,:]), pd.Series(rm_rf_paths[0,:]))
```

Kernel Regression function optimized for simulation. Compute only parameter estimates using passed in kernel.

```
#Kernel regression
def kernel_reg_sim(Y, X, kernel= lambda x:((2*np.pi)**-0.5)*np.exp(-0.5*(x**2)) ):
    """




    Perform Kernel Regression.
    Args:
        Y: array like, vector of response observations
        X: array like, vector of covariate observations
        h, float: bandwidth for Gaussian kernel.
        plot, boolean: flag to generate plots
        kernel, function: kernel function. Defaults to Gaussian.
    Returns:
        kernel_results, pd.DataFrame: DF containing parameter estimates, CIs, p-values, R2, MAPE, point APE
    Default kernel bandwidth is T^0.5. Literature suggests T^0.5 or T^0.4.
    """
    #Check for valid inputs
    if len(Y) != len(X):
        raise ValueError(f"Response vector Y length {len(Y)}, differs from covariate vector length {len(X)}.")

    #Reshape, prepend column of 1s to X
    Y = np.array(Y).reshape(len(Y),1)
    X = np.array(X).reshape(len(X),1)
    X = np.hstack((np.ones((len(X),1)), X))

    #Default behavior for kernel bandwidth
    h = len(Y)**0.5


    #Define index for kernel function
    i = np.arange(1,len(Y)+1)

    #per t, estimate beta, SE, p-val, R2, and point APE
    kernel_results = []
    for t in range(1, len(Y)+1):

        #compute weights
        w_t = kernel((t-i)/h)

        #W is a diagonal matrix with w_i at (i,i)
        W = np.diag(w_t)

        #Compute beta estimates
        try:
            beta = np.linalg.inv(X.T @ W @ X) @ (X.T @ W @ Y)
            b0, b1 = beta[0][0], beta[1][0]

        except np.linalg.LinAlgError as l:
            """
            Error at last value of t using custom kernel, because only (t,t) element of
```

```
            #weight matrix is non-zero. All else are 0, so reg. tries using 1 obs w/intercept
            and covariate hence Linear Algebra error
            print(f"Linear Algebra Error: {l} at t={t}.")

            #If so, set all values to nan
            b0 = np.nan
            b1 = np.nan

        #This segement executes regardless of error
        else:
            #Track results
            t_results = {"t":t,"B0":b0, "B1":b1}
            kernel_results.append(t_results)

    #Store as DF
    kernel_results = pd.DataFrame.from_dict(kernel_results)


    #Return dataframe
    return kernel_results
```

Testing

Utility function for error measures.

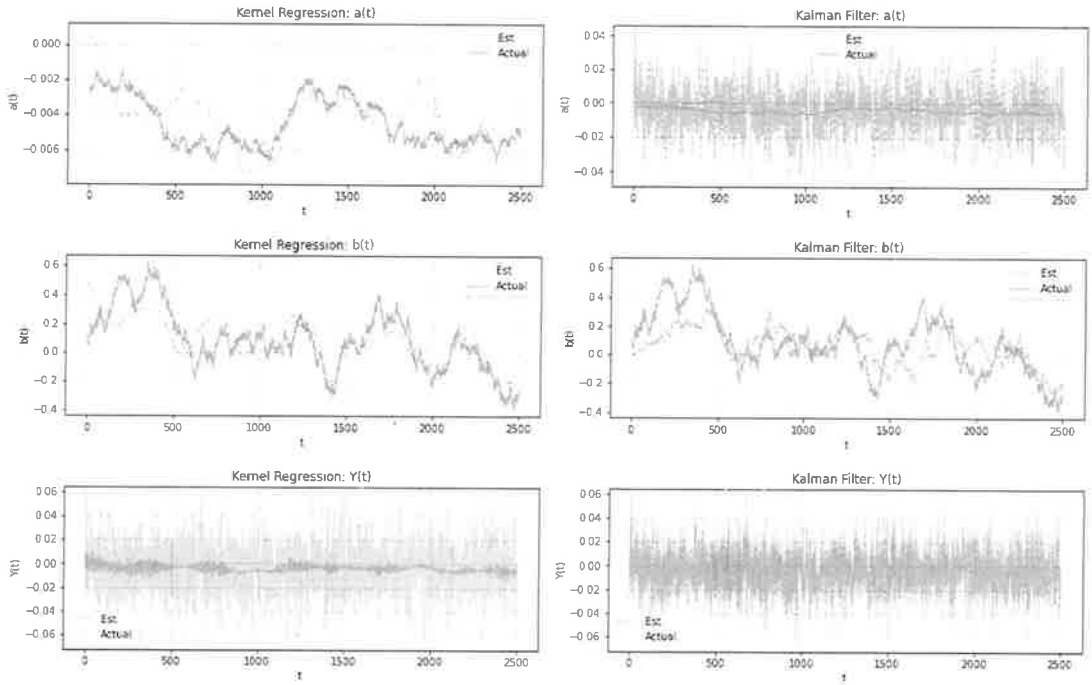
```
#Utility function for error metrics
def error_metrics(y_pred, y_true, label, plot=True):
    """
    Compute R2, MSE, MAPE. Plot.
    Args:
        y_pred: array, vector of predicted observations.
        y_true: array, vector of true observations.
        label: str, name of prediction method
        plot: boolean, flag to generate plots of true vs. predicted
    Returns:
        metrics: pd.DataFrame
    Use sklearn error measures. Plot residuals, co-movement, scatter.
    ACF of residuals and residuals squared for GARCH.
    """
    #Check same length
    try:
        assert(len(y_pred)==len(y_true))




    except AssertionError:
        print(f"Pred. length {len(y_pred)} does not equal actual length {len(y_true)}.")
        return
    #R2
    r2 = r2_score(y_true, y_pred)

    #RMSE
    rmse = mean_squared_error(y_true, y_pred)**0.5

    #MAPE
    mape = mean_absolute_percentage_error(y_true, y_pred)

    #Correlation
    r = np.corrcoef(y_pred, y_true)[0][1]

    #compute residuals
    resid = y_true - y_pred

    #Plot
    if plot:
        #4 subplots
        fig, axs = plt.subplots(nrows=3, ncols=2, figsize=(15,10))

        #Actuals vs. predicted
        axs[0,0].plot(y_true, label="Actual", linestyle="--")
        axs[0,0].plot(y_pred, label="Predicted", linestyle="--")
        axs[0,0].set_title("Actual vs. Predicted:"+r'$\rho$='+str(np.round(r, 2)))
        axs[0,0].grid()
        axs[0,0].legend()
        axs[0,0].set_xlabel("T")

        #Time series of residuals
        axs[0,1].plot(resid)
        axs[0,1].set_title("Residuals")
        axs[0,1].grid()
        axs[0,1].set_xlabel("T")

        #ACF residuals.
        sm.graphics.tsa.plot_acf(resid, lags=40, ax=axs[1,0])
        axs[1,0].grid()
        axs[1,0].set_title("Resid. ACF")

        #ACF of squared resid for ARCH effect.
        sm.graphics.tsa.plot_acf(resid**2, lags=40, ax=axs[1,1])
        axs[1,1].grid()
```

```
        axs[1,1].set_title("Resid. Sq. ACF")

        #Histogram residuals
        axs[2,0].hist(resid)
        axs[2,0].set_title("Resid. Dist. (e~N(0,1))")
        axs[2,0].grid()
        axs[2,0].axvline(x=np.mean(resid),
                         label=f"{np.round(np.mean(resid),2)}",
                         color="red", linestyle="--")
        axs[2,0].legend()


        #Scatter residuals
        axs[2,1].scatter(y_true, y_pred)
        axs[2,1].grid()
        axs[2,1].set_xlabel("Actual")
        axs[2,1].set_ylabel("Pred.")

        #Set title and display
        plt.suptitle(f"{label} Diagnostic Plots")
        plt.tight_layout()
        plt.show()

    #store results as DF
    metrics = {"R2":r2,"RMSE":rmse,"MAPE":mape,"Corr.":r,"T":len(y_true), "Sim:":label}
    return metrics


#Set number of simulations to test on. Large n and T is time consuming, large T critical for results.
test_i = n
if test_i > n:
    raise ValueError("Test sim. must be subset of total sim.")

#Choose a random sim to plot
plot_i = np.random.choice(range(test_i))



#For each sim, call estimation function, compute error measures.
kf_test_results = []
kernel_test_results = []
t0 = time.time()
for i in range(test_i):

    #Set Y_true and X_true





    Y_true = pd.Series(re_sim[i,:])
    X_true = pd.Series(rm_rf_paths[i,:])

    #True param values for a(t) and b(t)
    a_true = at_sim[i,:]
    b_true = bt_sim[i,:]

    #Kalman Filter results
    kf_i = kalman_filter_capm(Y_true, X_true)
    kf_a = kf_i['a(t)']
    kf_b = kf_i['b(t)']
    kf_y = kf_a + kf_b*X_true.values


    #Kernel reg results
    kernel_i = kernel_reg_sim(Y_true, X_true)
    kernel_a = kernel_i["B0"].values
    kernel_b = kernel_i["B1"].values
    kernel_y = kernel_a + kernel_b*X_true.values

    #Plot results for random sim
    if i==plot_i:
        t_idx = np.arange(1,T+1)
        fig, axs = plt.subplots(nrows=3, ncols=2, figsize=(15,10))

        #Kernel results a(t)
        axs[0][0].plot(t_idx, kernel_a, label="Est.", linestyle="--", alpha=0.5)
        axs[0][0].plot(t_idx, a_true, label="Actual.")
        axs[0][0].legend()
        axs[0][0].grid()
        axs[0][0].set_xlabel("t")
        axs[0][0].set_ylabel("a(t)")
        axs[0][0].set_title("Kernel Regression: a(t)")

        #Kernel results b(t)
        axs[1][0].plot(t_idx, kernel_b, label="Est.", linestyle="--", alpha=0.5)
        axs[1][0].plot(t_idx, b_true, label="Actual.")
        axs[1][0].legend()
        axs[1][0].grid()
        axs[1][0].set_xlabel("t")
        axs[1][0].set_ylabel("b(t)")
        axs[1][0].set_title("Kernel Regression: b(t)")

        #Kernel results y(t)
        axs[2][0].plot(t_idx, kernel_y, label="Est.", linestyle="--", alpha=0.5)
```

```
        axs[2][0].plot(t_idx, Y_true.values, label="Actual.", alpha=0.5)
        axs[2][0].legend()
        axs[2][0].grid()
        axs[2][0].set_xlabel("t")
        axs[2][0].set_ylabel("Y(t)")
        axs[2][0].set_title("Kernel Regression: Y(t)")

        #KF results a(t)
        axs[0][1].plot(t_idx, kf_a, label="Est.", linestyle="--", alpha=0.5)
        axs[0][1].plot(t_idx, a_true, label="Actual.")
        axs[0][1].legend()
        axs[0][1].grid()
        axs[0][1].set_xlabel("t")
        axs[0][1].set_ylabel("a(t)")
        axs[0][1].set_title("Kalman Filter: a(t)")

        #KF results b(t)
        axs[1][1].plot(t_idx, kf_b, label="Est.", linestyle="--", alpha=0.5)
        axs[1][1].plot(t_idx, b_true, label="Actual.")
        axs[1][1].legend()
        axs[1][1].grid()
        axs[1][1].set_xlabel("t")
        axs[1][1].set_ylabel("b(t)")
        axs[1][1].set_title("Kalman Filter: b(t)")

        #KF results y(t)
        axs[2][1].plot(t_idx, kf_y, label="Est.", linestyle="--", alpha=0.5)
        axs[2][1].plot(t_idx, Y_true.values, label="Actual.", alpha=0.5)
        axs[2][1].legend()
        axs[2][1].grid()
        axs[2][1].set_xlabel("t")
        axs[2][1].set_ylabel("Y(t)")
        axs[2][1].set_title("Kalman Filter: Y(t)")

        #Set title and display
        plt.suptitle(f"Simulation {i} Results:")
        plt.subplots_adjust(bottom=0.05, top=0.1)
        plt.tight_layout()
        plt.show()

    #Compute error measures
    kf_test_results.append(error_metrics(y_pred=kf_a, y_true=a_true, label="a(t)", plot=False))
    kf_test_results.append(error_metrics(y_pred=kf_b, y_true=b_true, label="b(t)", plot=False))
    kf_test_results.append(error_metrics(y_pred=kf_y, y_true=Y_true, label="y(t)", plot=False))
    kernel_test_results.append(error_metrics(y_pred=kernel_a, y_true=a_true, label="a(t)", plot=False))



    kernel_test_results.append(error_metrics(y_pred=kernel_b, y_true=b_true, label="b(t)", plot=False))
    kernel_test_results.append(error_metrics(y_pred=kernel_y, y_true=Y_true, label="y(t)", plot=False))

    #Time
    if i%100==0:
        print(f"{i}:{np.round(time.time()-t0,2)}s.")
        t0 = time.time()

#Check results
print("Kernel Regression:")
kernel_test_results = pd.DataFrame(kernel_test_results)
kernel_test_results.to_csv("kernel_test_results.csv")
display(kernel_test_results.groupby(["Sim:"])[["R2","MAPE"]].describe().round(3))

print("Kalman Filter Regression:")
kf_test_results = pd.DataFrame(kf_test_results)
kf_test_results.to_csv("kalman_filter_results.csv")
display(kf_test_results.groupby(["Sim:"])[["R2","MAPE"]].describe().round(3))
```

```
0:57.66s.
100:5230.29s.
```

Simulation 108 Results:




```
200:4947.53s.
300:5608.87s.
400:5048.26s.
500:5038.77s.
600:5058.93s.
700:5024.91s.
800:5085.86s.
900:5037.19s.
Kernel Regression:
```

	R2								MAPE							
	count	**mean**	**std**	**min**	**25%**	**50%**	**75%**	**max**	**count**	**mean**	**std**	**min**	**25%**	**50%**	**75%**	**max**
Sim:																
a(t)	1000.0	0.131	0.767	-5.910	-0.167	0.363	0.642	0.960	1000.0	6.268	96.845	0.085	0.423	1.466	3.648	3056.513
b(t)	1000.0	0.748	0.216	-0.444	0.666	0.812	0.900	0.982	1000.0	2.182	11.628	0.044	0.273	0.907	2.048	348.150
y(t)	1000.0	0.180	0.128	0.025	0.081	0.139	0.241	0.706	1000.0	4.177	33.536	1.286	2.014	2.427	3.153	1055.376

Kalman Filter Regression:

	R2								MAPE							
	count	**mean**	**std**	**min**	**25%**	**50%**	**75%**	**max**	**count**	**mean**	**std**	**min**	**25%**	**50%**	**75%**	**max**
Sim:																
a(t)	1000.0	-61.215	48.879	-341.438	-82.495	-46.260	-27.036	-3.575	1000.0	89.689	1976.556	0.797	3.646	13.890	30.953	62502.666
b(t)	1000.0	0.521	0.409	-2.866	0.380	0.639	0.793	0.967	1000.0	2.073	3.772	0.114	0.332	1.057	2.482	64.630
y(t)	1000.0	0.824	0.027	0.786	0.803	0.815	0.837	0.935	1000.0	3.635	45.232	0.853	1.395	1.654	2.127	1429.438

Kernel regression implementation is very slow with large T. It requires fitting NT regressions (e.g. 2.5mm regressions for N=1000, T=2500).